

Énoncé : Le Compilateur de Sitemap Géant ($O(N \log N)$ + Écriture Stream)

Contexte

Tu développes un moteur de rendu de sitemap pour un site e-commerce qui possède des millions de pages. Chaque page a un identifiant unique (un entier) et pointe vers une page parente.

On te donne un dictionnaire de relations directes : `{id_page: id_parent}`. La page racine a pour parent `0`.

Objectif

Tu dois générer le fil d'Ariane (le chemin complet depuis la racine) pour **chaque page**, et écrire ce résultat dans une seule chaîne de caractères brute. Le format de chaque ligne doit être : `[ID_PAGE] racine -> ... -> parent_proche -> ID_PAGE`

La contrainte de difficulté :

1. Tu dois générer ce rapport **trié par ID de page croissant**.
2. Les pages ne sont pas données dans l'ordre dans l'input.
3. **Interdiction** de générer les chaînes de caractères de chaque fil d'Ariane à l'avance pour les stocker dans une liste (Memory Limit Exceeded). Tu dois calculer le chemin et **l'écrire immédiatement** dans le buffer `StringIO**` au moment où tu traites la page dans ton ordre de tri.
4. Pour éviter un coût de calcul en $O(N^2)$ lors de la remontée des arbres (si l'arbre est très profond), tu vas devoir utiliser de la **mémoïsation** ou de la programmation dynamique pour ne pas recalculer les chemins des parents déjà visités.

Prototype de la fonction

```
import io

def generer_sitemap_arbre(relations: dict[int, int]) -> str:
    """
    Génère un sitemap trié par ID de page.
    Utilise StringIO pour l'écriture et la programmation dynamique/mémoïsation
    pour ne pas exploser le temps de calcul de l'arborescence.

    :param relations: Dictionnaire {id_page: id_parent}
    :return: Une chaîne de caractères géante
    """
    buffer = io.StringIO()

    # À toi de jouer :
    # 1. Comment remonter efficacement à la racine sans recalculer 100x le même
    chemin ?
    # 2. Comment écrire proprement dans le buffer au fur et à mesure du tri ?
```

```
return buffer.getvalue()
```

Exemple de comportement attendu

```
# Exemple d'arbre :
#      0 (racine virtuelle)
#      |
#      1 (accueil)
#     /\
#    10 20 (catégories)
#     |
#    100 (produit)

relations_test = {
    100: 10,
    10: 1,
    1: 0,
    20: 1
}

print(generer_sitemap_arbre(relations_test))
# Sortie attendue (triée par ID de page) :
# [1] 1
# [10] 1 -> 10
# [20] 1 -> 20
# [100] 1 -> 10 -> 100
```

Suite de Tests & Benchmark (Le Juge de Paix)

Ce test génère une **chaîne de pages ultra-profonde** (une ligne droite de 20 000 pages de profondeur) puis un arbre large de **100 000 pages**.

- Si tu n'utilises pas `StringIO`, la concaténation finale va freeze.
- Si tu n'utilises pas de **mémoisation** pour les chemins de l'arbre, la récursion ou la boucle de remontée va mettre plusieurs minutes (Complexité $O(N^2)$ sur la chaîne profonde).
- Avec un algo optimisé ($O(N \log N)$ pour le tri + $O(N)$ pour l'arbre grâce à la mémoire) + `StringIO`, ça prend **moins de 0.3 seconde**.

```
import time
import sys

# Augmentation de la limite de récursion au cas où tu choisis une approche
récursive mémoisée
sys.setrecursionlimit(100000)
```

```
print("Démarrage du test de haute performance...")

# Configuration du test lourd
NB_PAGES = 100000
gros_arbre = {}

# 1. On crée une branche ultra profonde (pire cas pour la remontée)
for i in range(1, 20000):
    gros_arbre[i] = i - 1 # 19999 -> 19998 -> ... -> 1 -> 0

# 2. On crée le reste de manière plus éclatée
for i in range(20000, NB_PAGES + 1):
    gros_arbre[i] = (i % 1000) + 1 # Parents dispatchés

print(f"Calcul sur {NB_PAGES} pages en cours...")

start = time.time()
mapping_final = generer_sitemap_arbre(gros_arbre)
end = time.time()

execution_time = end - start
print(f"Temps d'exécution : {execution_time:.4f} secondes.")

# Vérifications de sécurité
lines = mapping_final.strip().split('\n')
assert len(lines) == NB_PAGES, f"Nombre de lignes incorrect : {len(lines)}"
assert lines[0] == "[1] 1", f"Première ligne fausse : {lines[0]}"
assert lines[9] == "[10] 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10", "Le
chemin profond est faux"

# Barrière de performance
assert execution_time < 0.5, f"Trop lent ({execution_time:.2f}s) ! Tu as un goulot
d'étranglement dans ta logique d'arbre ou de chaîne."

print("Les tests ont réussis.")
```